EE1D1: Digital Systems A

BSc. EE, year 1, 2026-2027, lecture 5

Combinational Modules and Implementation Technology

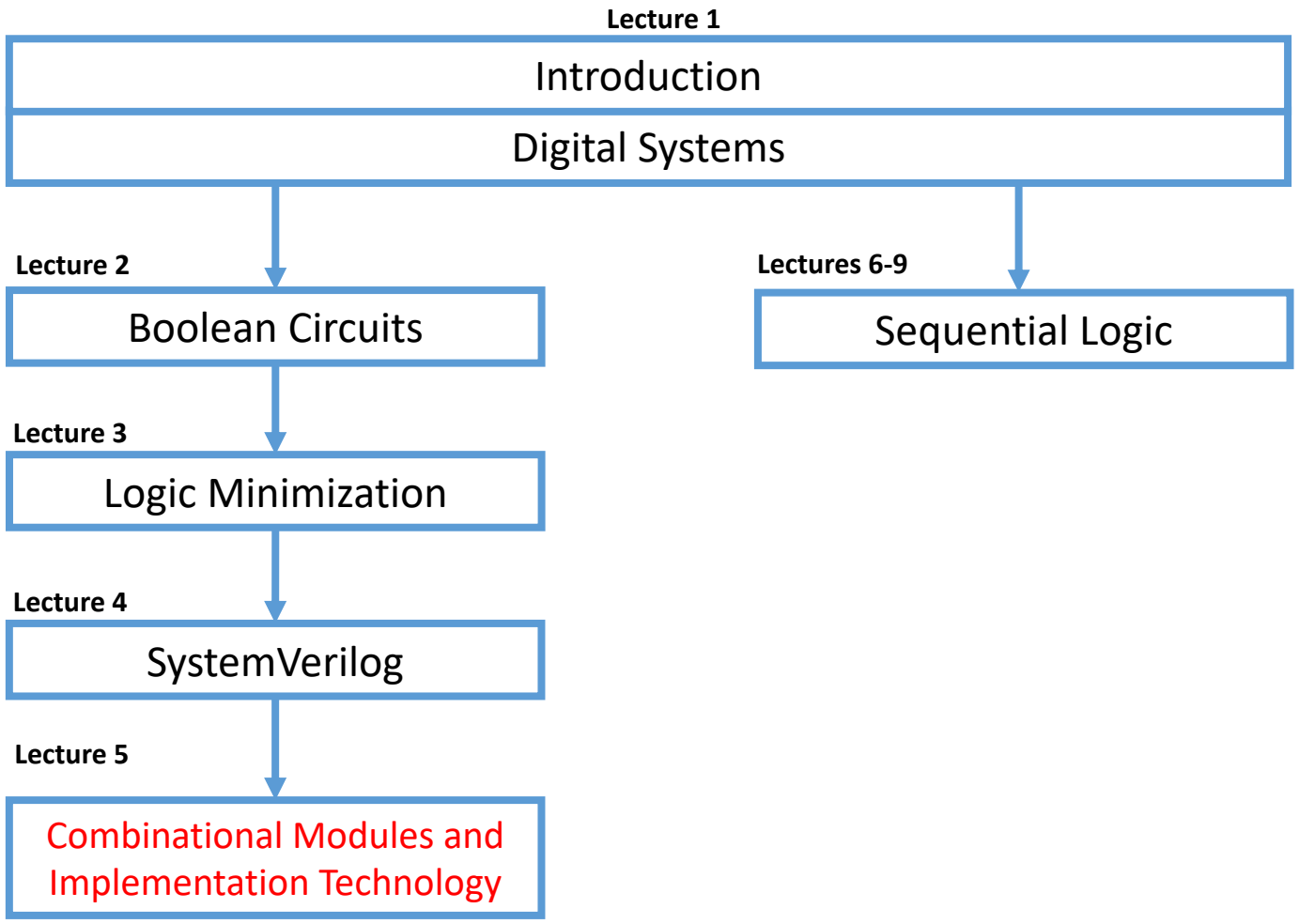
Computer Engineering Lab

Faculty of Electrical Engineering, Mathematics & Computer Science

TU Delft

Delft University of Technology

Recap


Lecture 1
Introduction
Digital Systems
Lecture 2
Boolean Circuits
Lecture 3
Logic Minimization
Lecture 4
SystemVerilog
Lecture 5
Combinational Modules and Implementation Technology
Lectures 6-9
Sequential Logic

Recap

week	Monday	Tuesday	Wednesday	Thursday	Friday
1		Introduction to Digital Systems			seminar 1
2	Boolean Circuits	Logic Minimization			seminar 2
3	Intro SystemVerilog	seminar 3		course lab 1	
4	Combinational Modules and Implementation Technology	seminar 4	Test 1 (trial exam) (11:00 – 12:00)	course lab 2	seminar 5
5	Exam 1 Friday Oct. 2 (9:00 – 11:00)				

Every lecture comes with a set of lecture assignments. Finish them before the next lecture.

Outline

- Combinational Modules
  - Multiplexers
  - Demultiplexer
  - Decoders
  - Encoders
- CMOS Implementation Technology
  - NMOS and PMOS transistors
  - Logic gates
  - Buffers

Sections in book: 1.6, 1.7.4 - 1.7.7 and 2.8

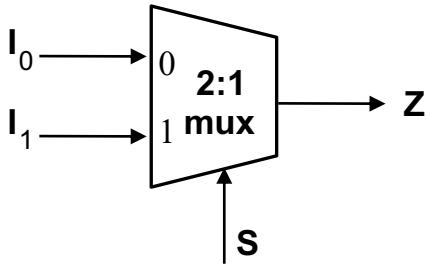
Learning Objectives

As student you should be able to:

- Use combinational modules to implement Boolean functions
- Implement Boolean gates and circuits with CMOS transistors

Combinational Modules – Multiplexers

- Multiplexer (also called Selector):
  - Select one of several inputs and transmit the selected input signal to the output
  - With an *n*-bit control signal, a selection can be made between 2^n inputs



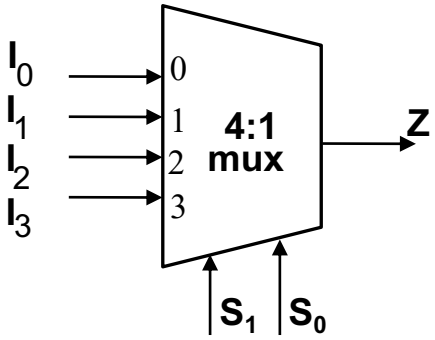
Truth Table

S	Z
0	I_0
1	I_1

$Z = S' I_0 + S I_1$

Truth Table

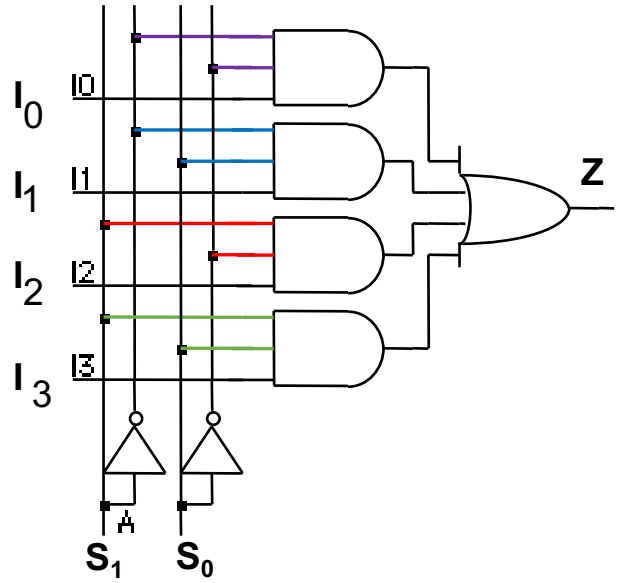
I_1	I_0	S	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



S_1	S_0	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$Z = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$

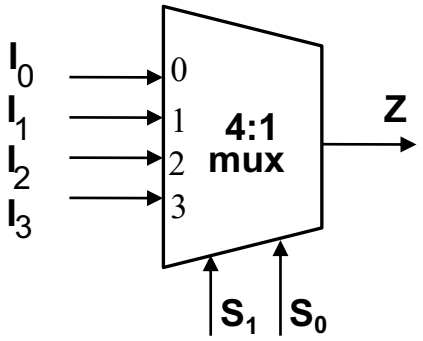
Possible implementation



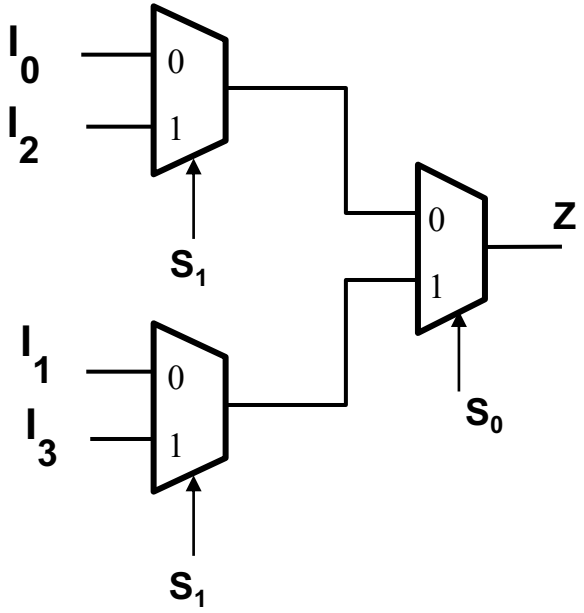
Combinational Modules – Multiplexers

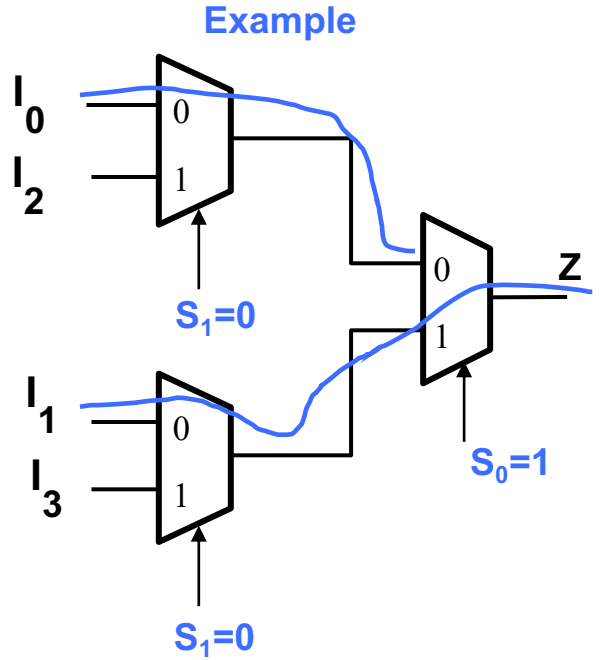
- Multiplexer:
  - Select one of several inputs and transmit the selected input signal to the output
  - With an *n*-bit control signal, a selection can be made between 2^n inputs
  - *Larger multiplexers can be created by combining smaller ones*

A 4-to-1 multiplexer implemented with three 2-to-1 multiplexers



S_1	S_0	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3





Combinational Modules – Multiplexers – Logic Functions

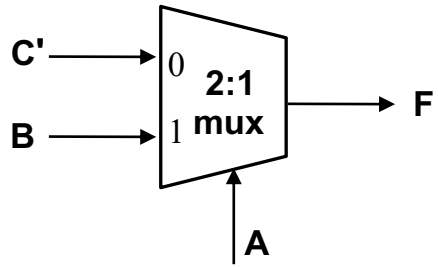
Example: $F(A,B,C) = m_0 + m_2 + m_6 + m_7$

$= A'\,B'\,C' + A'\,B\,C' + A\,B\,C' + A\,B\,C$

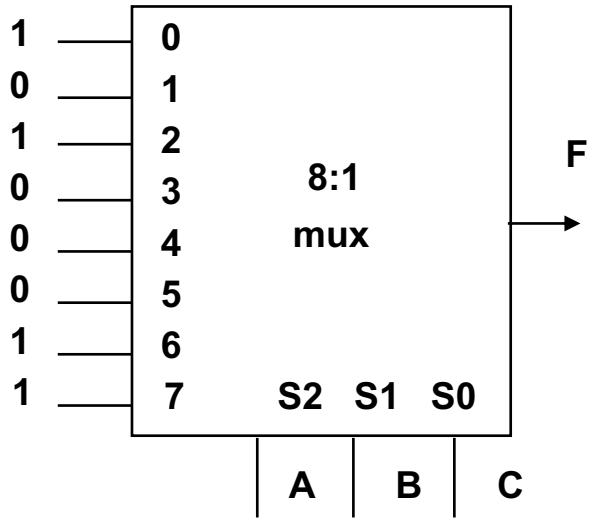
$= A'\,B'\,C'\,(1) + A'\,B'\,C\,(0) + A'\,B\,C'\,(1) + A'\,B\,C\,(0)$
$A\,B'\,C'\,(0) + A\,B'\,C\,(0) + A\,B\,C'\,(1) + A\,B\,C\,(1)$

$A'\,B'\,C' + A'\,B\,C' + A\,B\,C' + A\,B\,C$
$= A'\,B'\,(C') + A'\,B\,(C') + A\,B\,(C'+C)$
$= A'\,B'\,(C') + A'\,B\,(C') + A\,B\,(1)$
$= A'\,B'\,(C') + A'\,B\,(C') + A\,B'\,(0) + A\,B\,(1)$

$F = A'\,C' + A\,B$



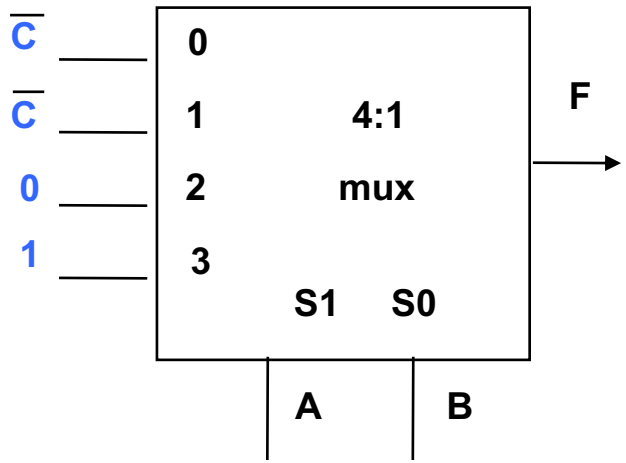
2^n:1 MUX for *n* variables



"Lookup Table"

A	B	C	F	
0	0	0	1	$\overline{C}$
0	0	1	0	
0	1	0	1	$\overline{C}$
0	1	1	0	
1	0	0	0	0
1	0	1	0	
1	1	0	1	1
1	1	1	1	

2^{n-1}:1 MUX for *n* variables



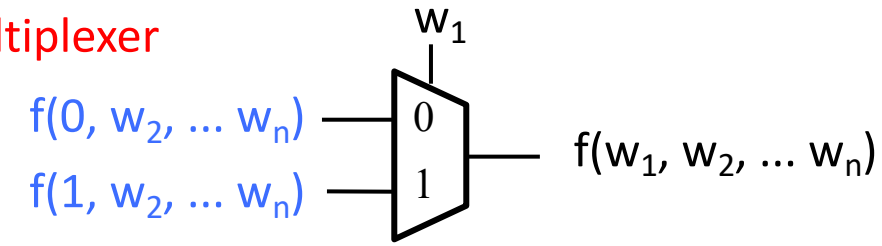
Combinational Modules – Multiplexers – Shannon's Expansion Theorem

Shannon's theorem: Every arbitrary Boolean function

$f(w_1, w_2, \ldots w_n)$ can be written in the form:

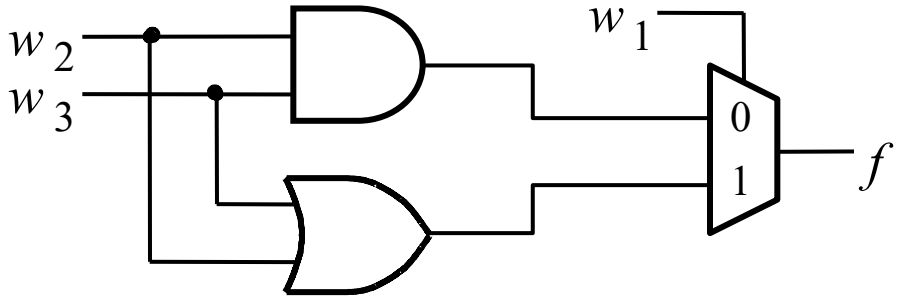
$f(w_1, w_2, \ldots w_n) = w_1' \cdot f(0, w_2, \ldots w_n) + w_1 \cdot f(1, w_2, \ldots w_n)$

Implementation using a multiplexer



Example

$$f(w_1, w_2, w_3) = w_1' w_2 w_3 + w_1 w_2' w_3 + w_1 w_2 w_3' + w_1 w_2 w_3$$
$$= w_1' f(0, w_2, w_3) + w_1 f(1, w_2, w_3)$$
$$= w_1' (w_2 w_3) + w_1 (w_2' w_3 + w_2 w_3' + w_2 w_3)$$
$$= w_1' (w_2 w_3) + w_1 (w_2 + w_3)$$



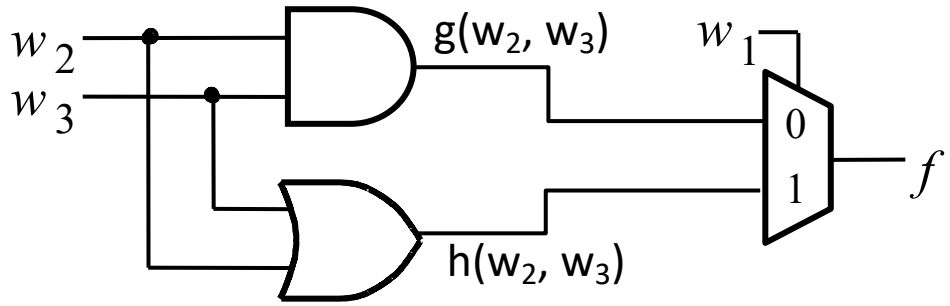
Note that $f(0, w_2, \ldots w_n)$ and $f(1, w_2, \ldots w_n)$ can be simplified recursively until the circuit consists only of multiplexers of the required size.

Combinational Modules – Multiplexers – Shannon's Expansion Theorem

$f(w_1, w_2, w_3) = w_1' (w_2\, w_3) + w_1 (w_2 + w_3)$

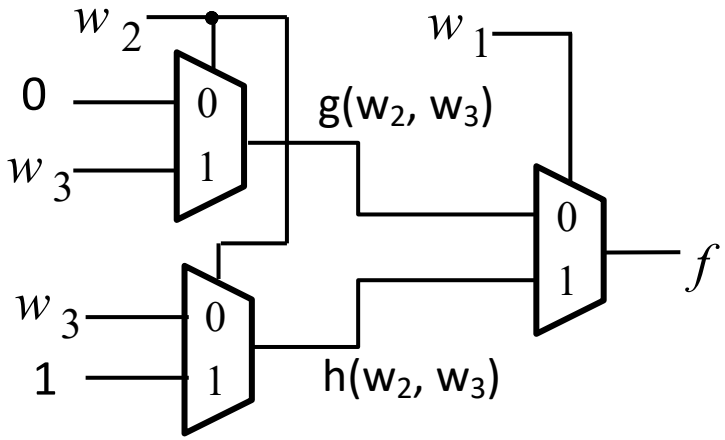
$= w_1' \, g(w_2, w_3) + w_1 \, h(w_2, w_3)$

with $g(w_2, w_3) = w_2\, w_3$ and $h(w_2, w_3) = w_2 + w_3$



$g(w_2, w_3) = w_2' \, g(0, w_3) + w_2 \, g(1, w_3) = w_2' \, 0 + w_2 \, w_3$

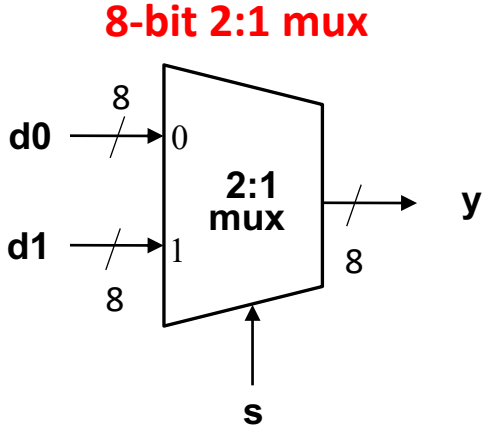
$h(w_2, w_3) = w_2' \, h(0, w_3) + w_2 \, h(1, w_3) = w_2' \, w_3 + w_2 \, 1$



Combinational Modules – Multiplexers –SystemVerilog

2:1 mux for 8-bit data vectors:

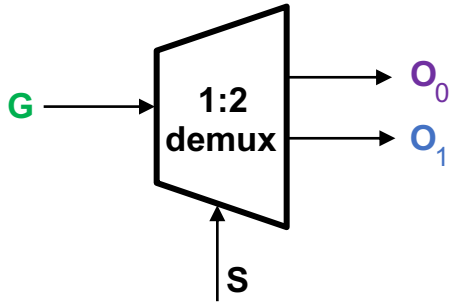
```
module mux2
    (input  logic [7:0] d0, d1,
     input  logic s,
     output logic [7:0] y);
  assign y = s ? d1 : d0;
endmodule
```



Combinational Modules – Demultiplexer

- Demultiplexer (also called Deselector):
  - Activate one of the 2^n output bits using n control bits
  - The selected output bit takes the value of the input

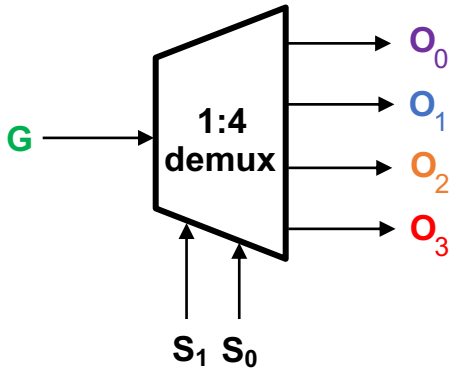
1:2 Demux



S	O_1	O_0
0	0	G
1	G	0

$O_0 = G\ S'$
$O_1 = G\ S$

1:4 Demux



$O_0 = G\ S_1'\ S_0'$

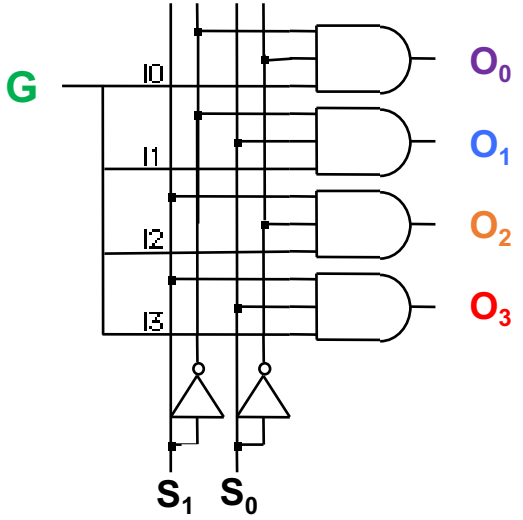
$O_1 = G\ S_1'\ S_0$

$O_2 = G\ S_1\ S_0'$

$O_3 = G\ S_1\ S_0$

S_1	S_0	O_3	O_2	O_1	O_0
0	0	0	0	0	G
0	1	0	0	G	0
1	0	0	G	0	0
1	1	G	0	0	0

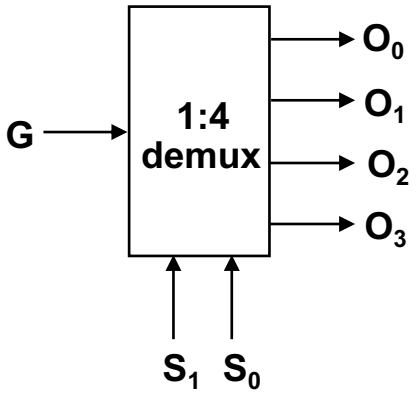
Possible implementation



Combinational Modules – Decoder

- Decoder:
  - Demultiplexer where the input G does not represent data but an enable signal E (or is always 1) and S is the incoming data.
  - This $1:2^n$ demultiplexer is called a $n:2^n$ decoder

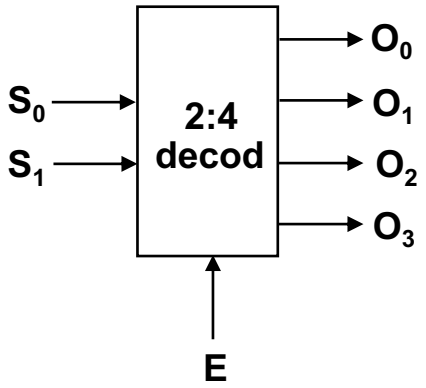
$1:2^2$ demultiplexer



S_1	S_0	O_3	O_2	O_1	O_0
0	0	0	0	0	G
0	1	0	0	G	0
1	0	0	G	0	0
1	1	G	0	0	0

implementation can be the same

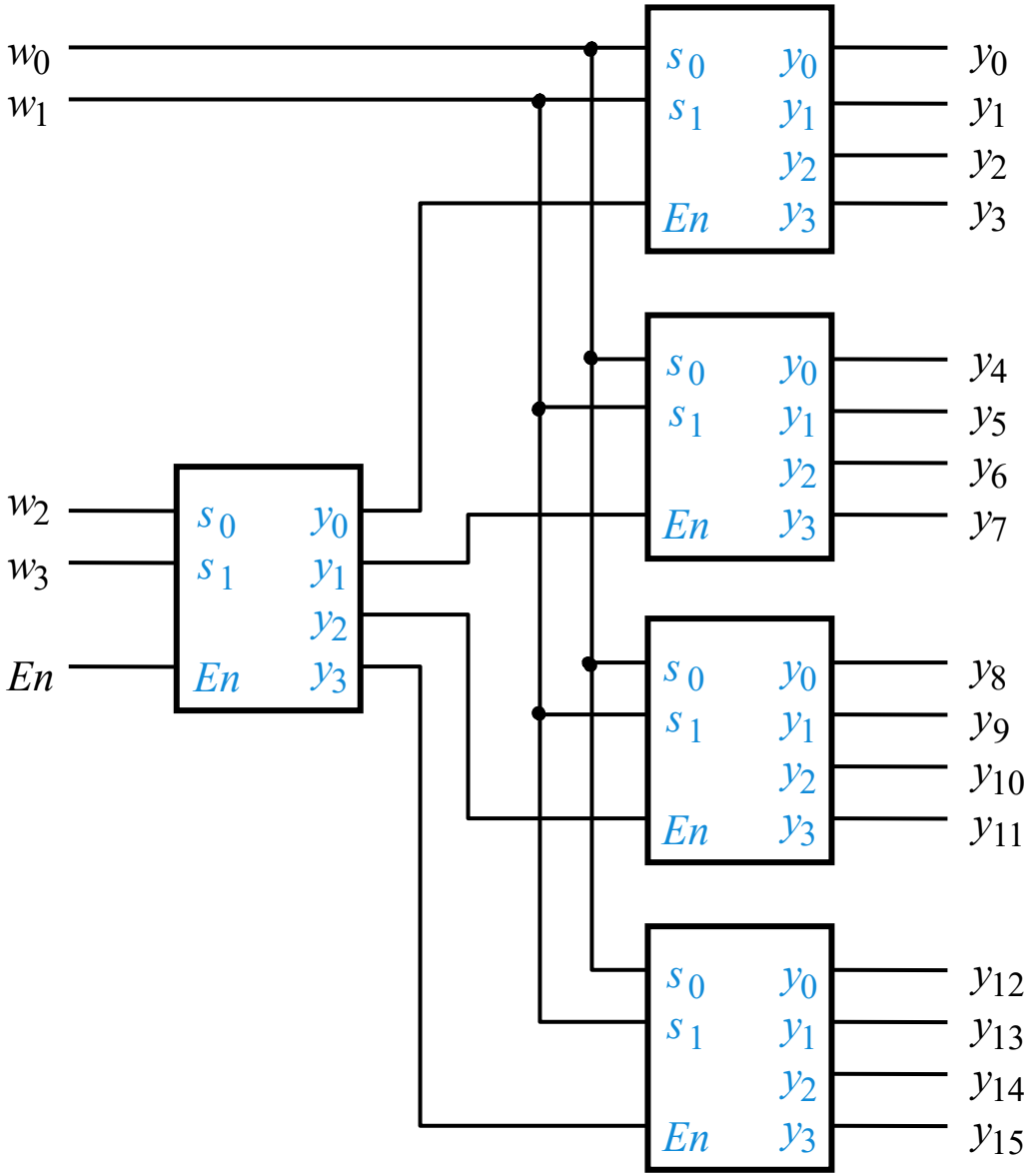
$2:2^2$ decoder



S_1	S_0	O_3	O_2	O_1	O_0
0	0	0	0	0	E
0	1	0	0	E	0
1	0	0	E	0	0
1	1	E	0	0	0

Combinational Modules – Decoder – Decoder Tree

A 4-to-16 decoder built from five 2-to-4 decoders with enable input

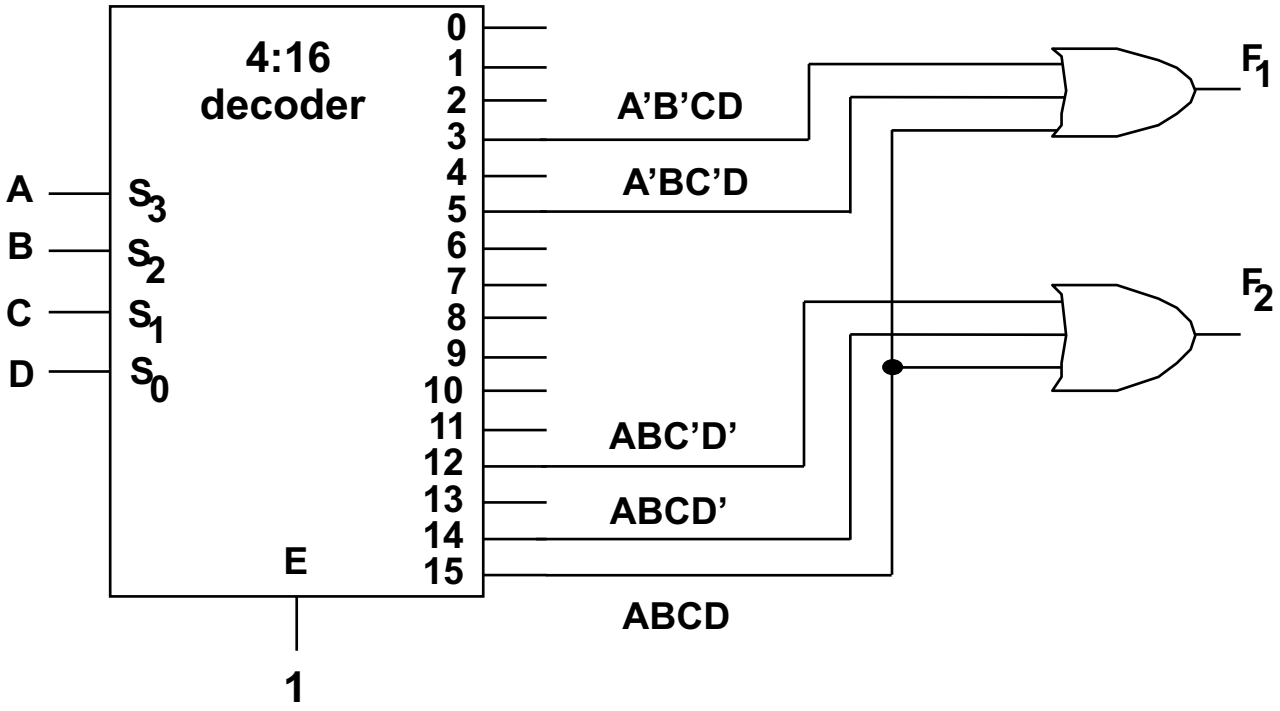


Combinational Modules – Decoder – Logic Functions

- Logic Functions:
  - A decoder together with OR gates can also be used to create logic functions.
  - The decoder provides the minterms, an OR gate creates a sum of minterms.

Examples: $F_1 = A'\,B'\,C\,D + A'\,B\,C'\,D + A\,B\,C\,D = m3 + m5 + m15$

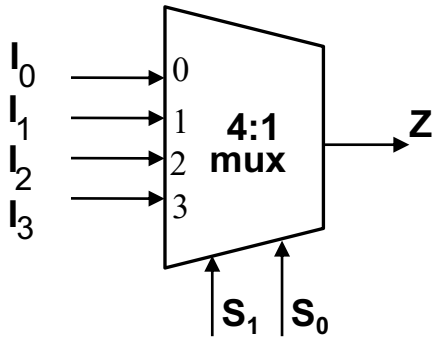
$F_2 = A\,B\,C'\,D' + A\,B\,C = A\,B\,C'\,D' + A\,B\,C\,D + A\,B\,C\,D' = m12 + m14 + m15$



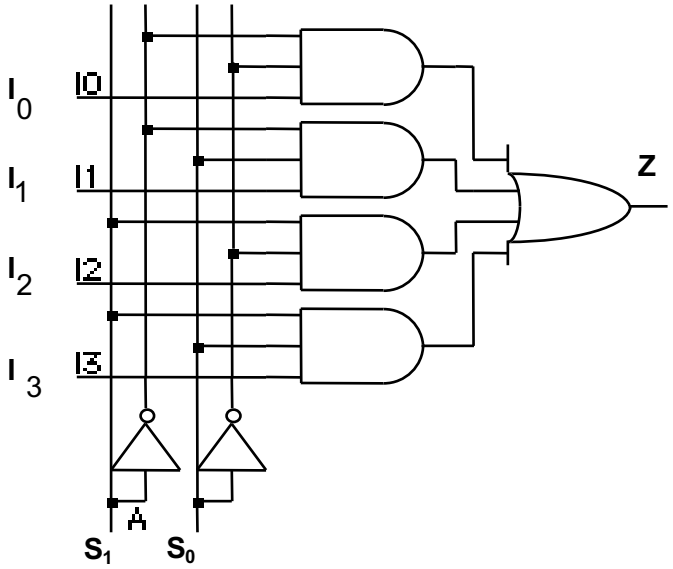
Combinational Modules – Decoder – Multiplexer Impl.

- Multiplexer:
  - Can be partially implemented with a decoder

MUX

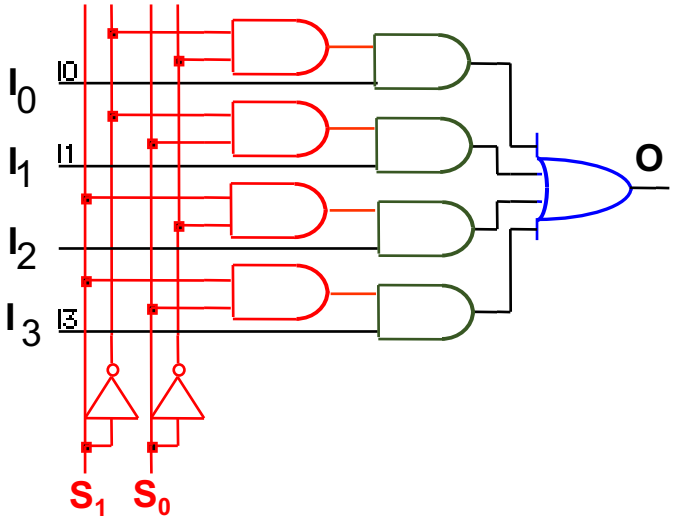


S_1	S_0	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



$Z = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$

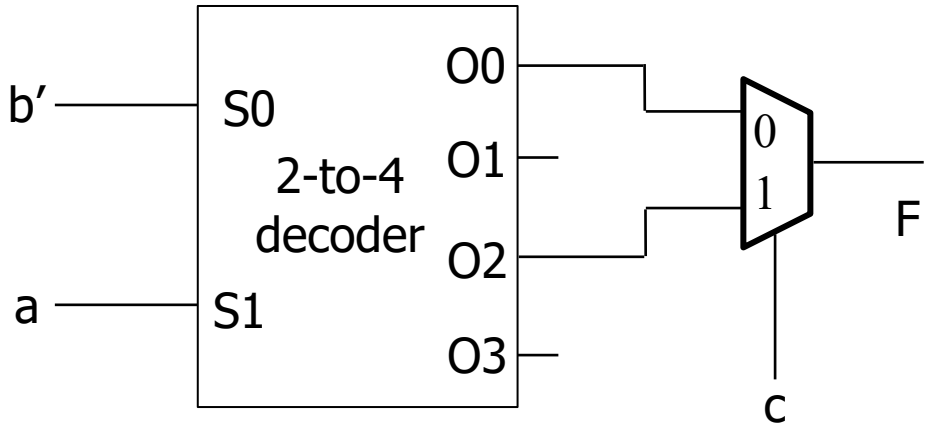
MUX composed of a decoder with E=1 and a AND-OR circuit



Combinational Modules – Question 1

Which logic function is implemented by the following circuit ?

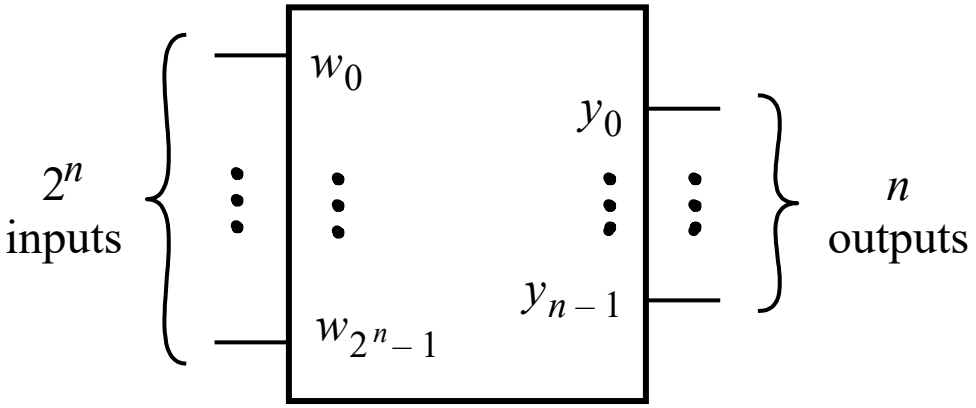
a. F = a′ b′ c + a b c′
b. F = a′ b c′ + a b c
c. F = a′ b′ c′ + a b c
d. A correct answer is not listed above



F = O0 c′ + O2 c
= (S1′ S0′) c′ + (S1 S0′) c
= (a′ (b′)′) c′ + (a (b′)′) c
= a′ b c′ + a b c
Hence answer b

Combinational Modules – Encoder

- Encoder:
  - Inverse function of decoder
  - 2^n inputs are mapped to n outputs



w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Supposes one-hot encoding at input

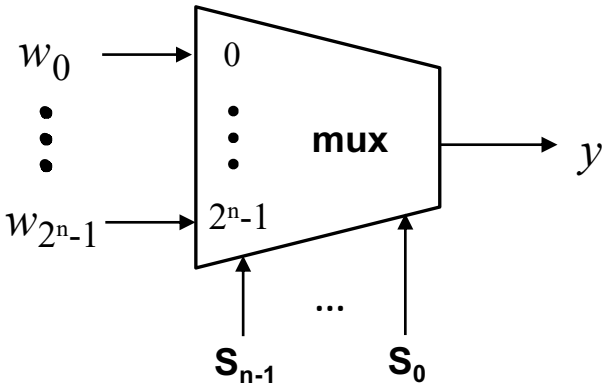
Priority Encoder

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

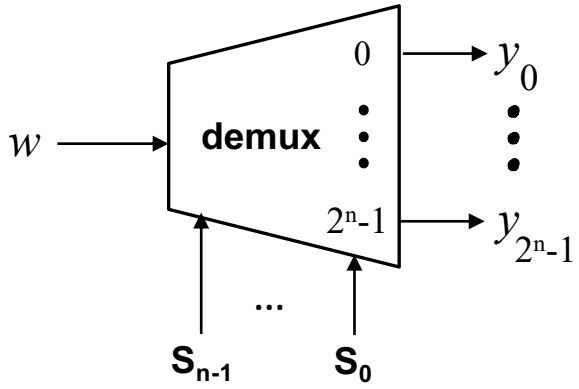
Now outputs defined for all input combinations.
z indicates if there is a 1 in the input at all.

Combinational Modules – Summary

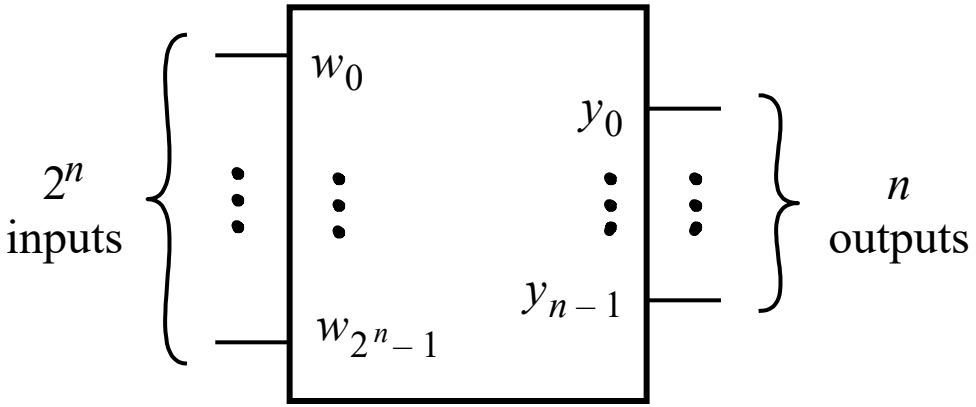
Multiplexer: 2^n:1 with n control signals



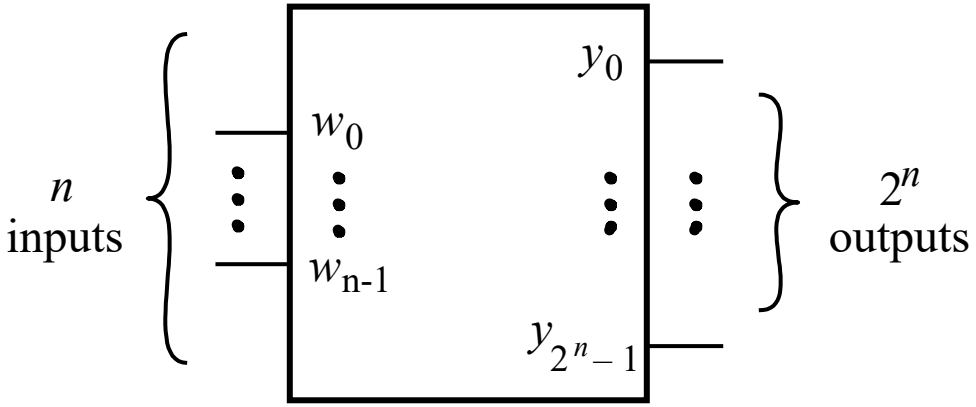
Demultiplexer: 1:2^n with n control signals



Encoder: 2^n:n with optionally an enable signal



Decoder: n:2^n with optionally an enable signal

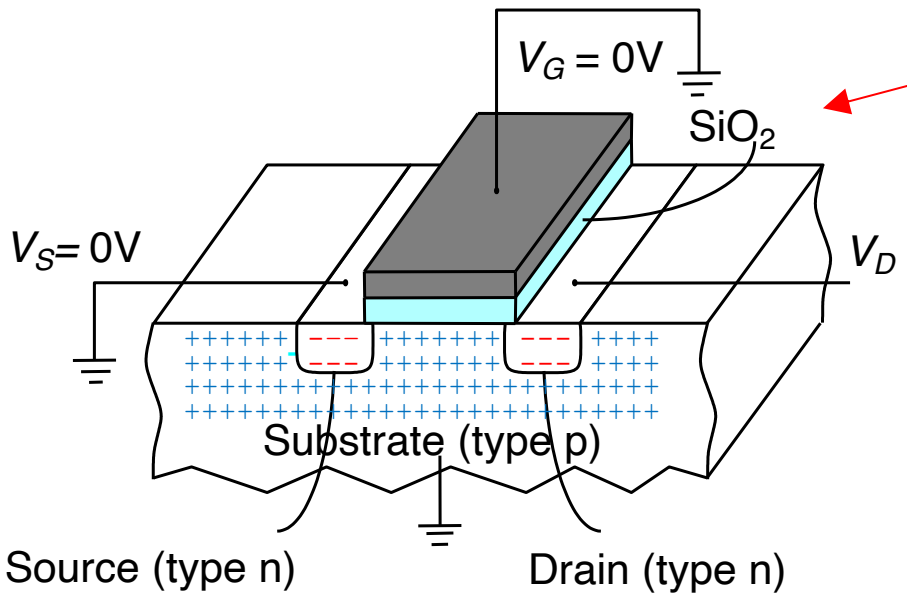


EE1D1: Digital Systems A

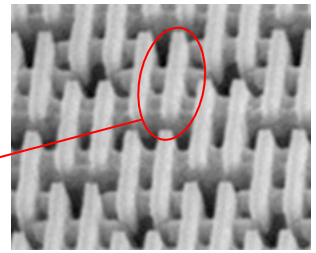
CMOS Implementation

CMOS Implementation Technology – The MOS Transistor

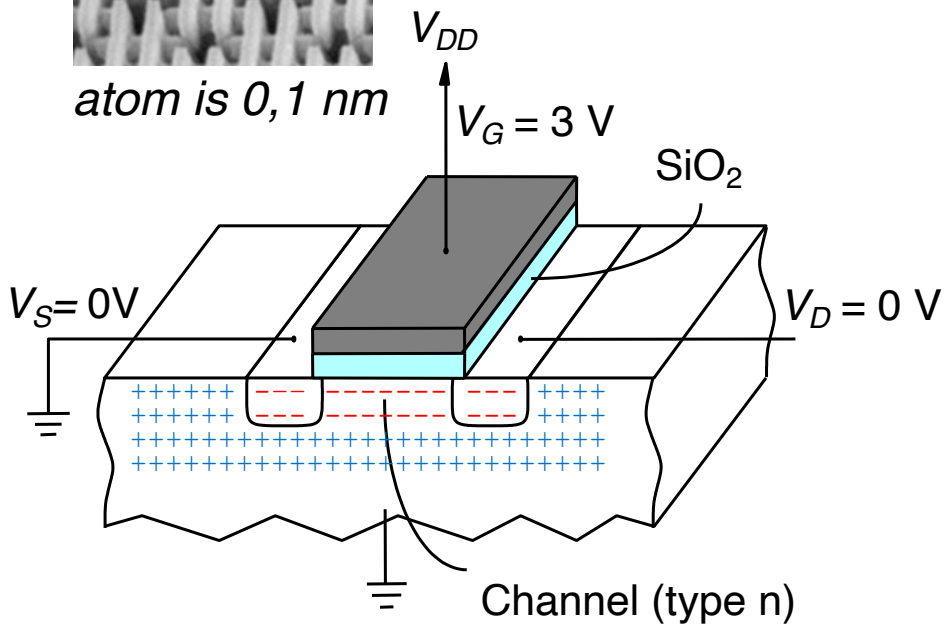
- CMOS = Complementary MOS (NMOS and PMOS transistors)
- Operation NMOS transistor:



dimensions
≈ 5 – 180 nm



atom is 0,1 nm



V_G = 0 V : transistor is off

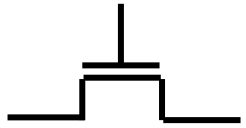
V_G = 3 V : transistor is on

- Operation PMOS transistor (p and n regions swapped):

V_G = 0 V : transistor is on V_G = 3 V : transistor is off

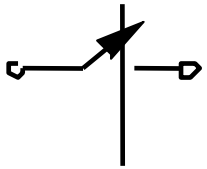
CMOS Implementation Technology – The MOS Transistor

The "digital abstraction" of an NMOS and a PMOS transistor

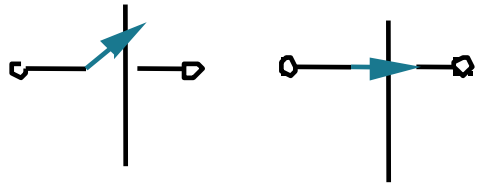


nmos transistor

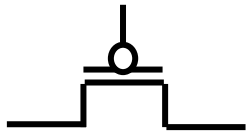
Normally Open:



True: closed
False: open

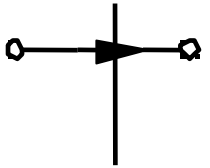


0 **1**

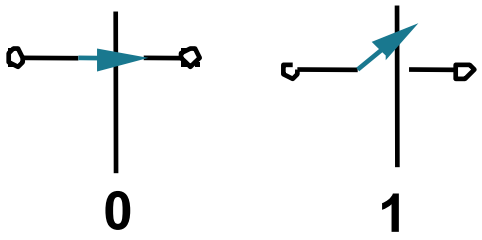


pmos transistor

Normally Closed:



True: open
False: closed

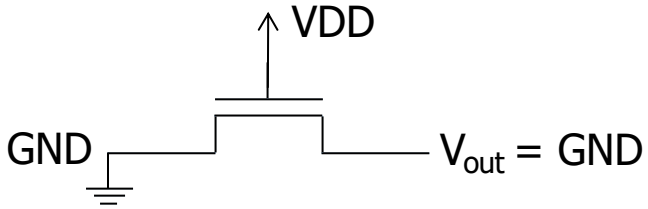


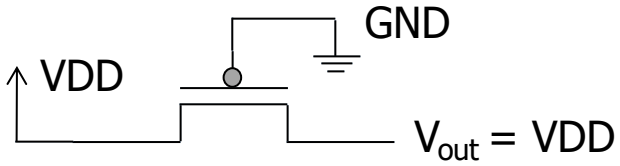
0 **1**

CMOS Implementation Technology – Pass Transistors

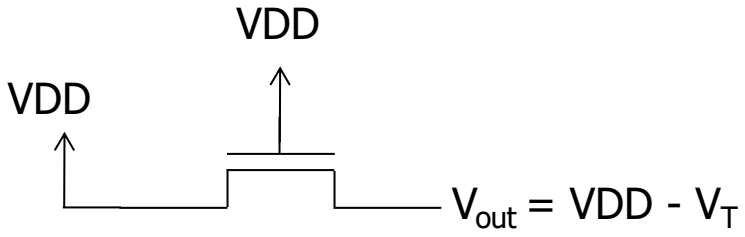
NMOS and PMOS Transistors can be used as switches to make connections and propagate a 0 or 1.

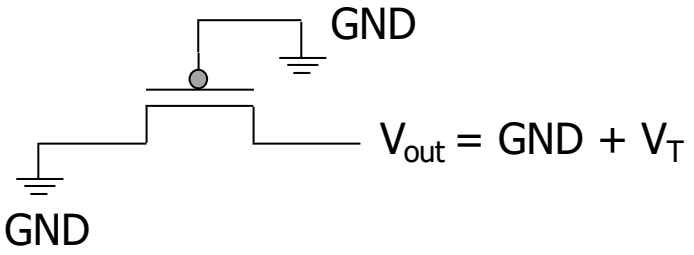
Ok is:



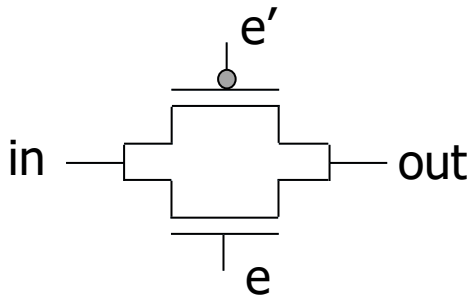


But what does not work well (V_T is threshold voltage for "on" mode, typically 0.3 – 0.7 V):

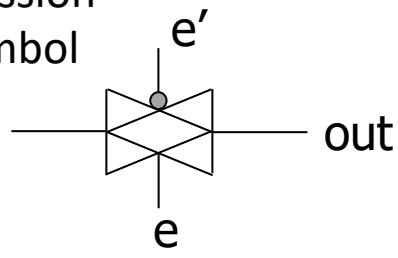




Solution: transmission gate.
Propagates both GND (0) and VDD (1) without loss when e = 1

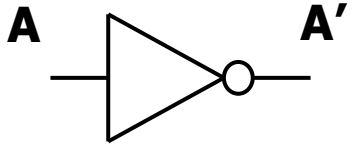


Transmission gate symbol

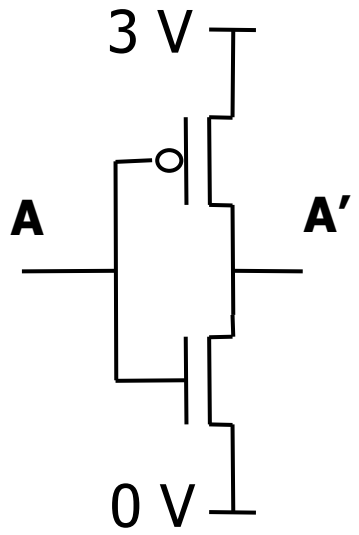


CMOS Implementation Technology – CMOS Inverter

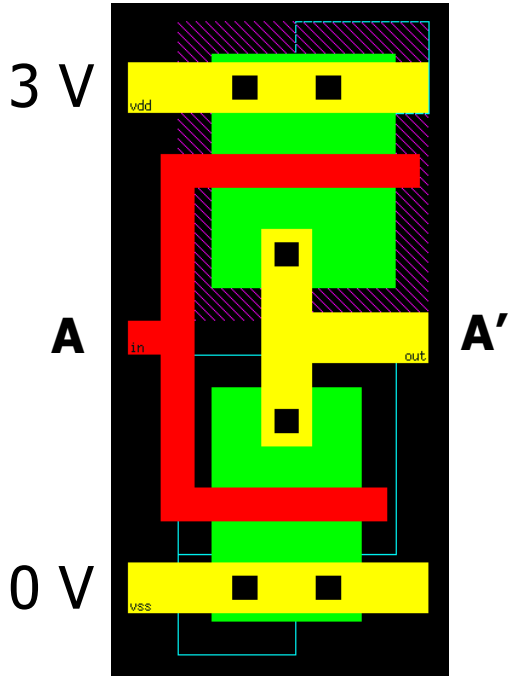
Logic:



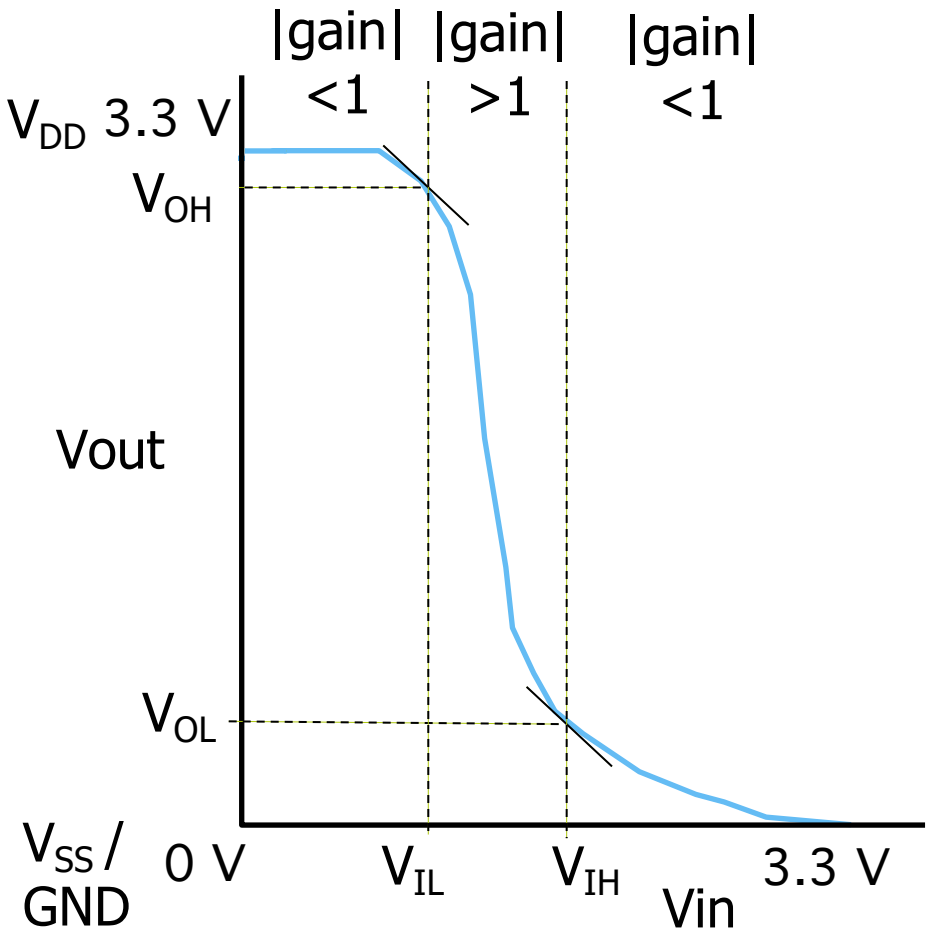
Electrical



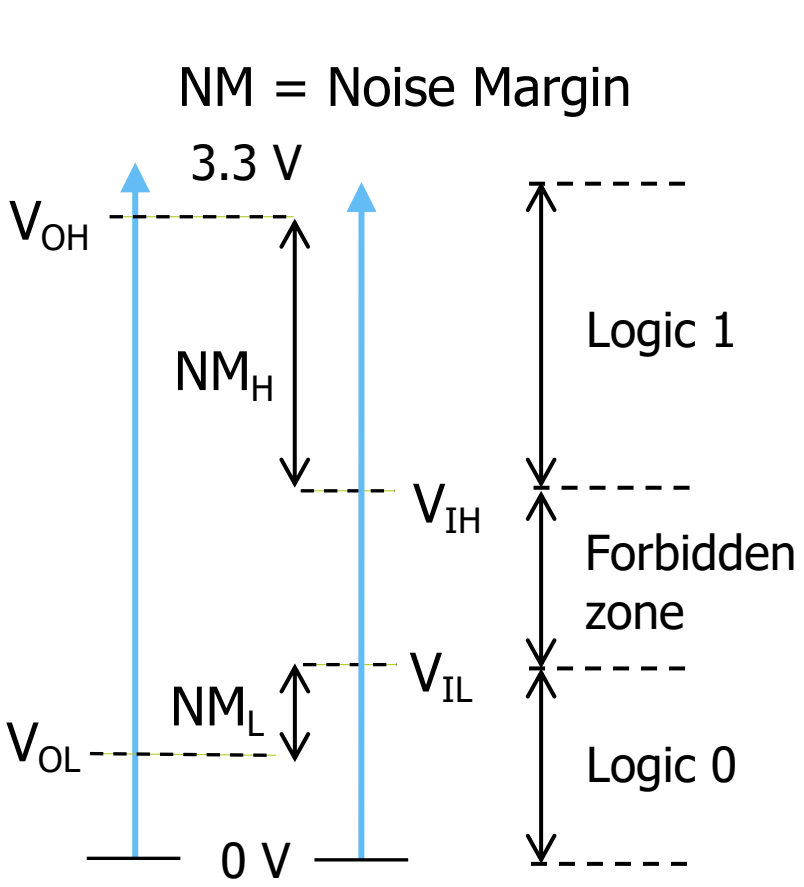
Layout

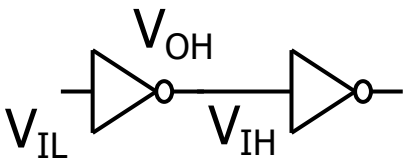


CMOS Implementation Technology – Behaviour Inverter



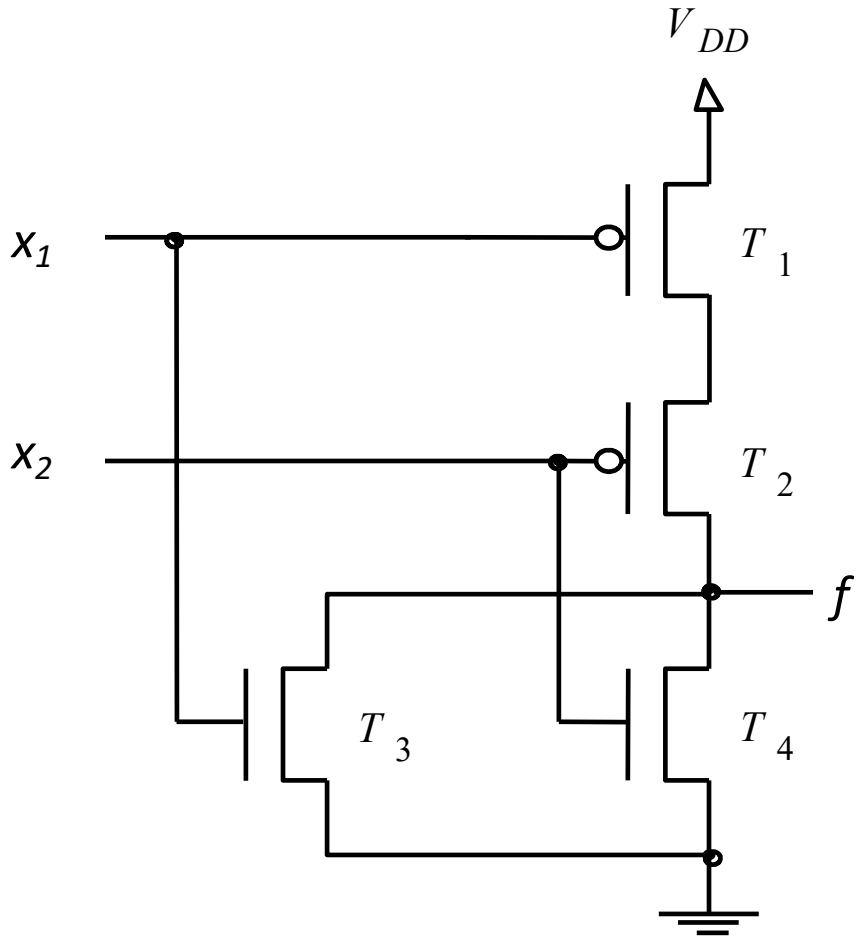
Although we use digital abstraction, real behaviour is analogue





Above V_{IH} and below V_{IL}, disturbances in the input voltage will not be amplified

CMOS Implementation Technology – CMOS NOR Gate

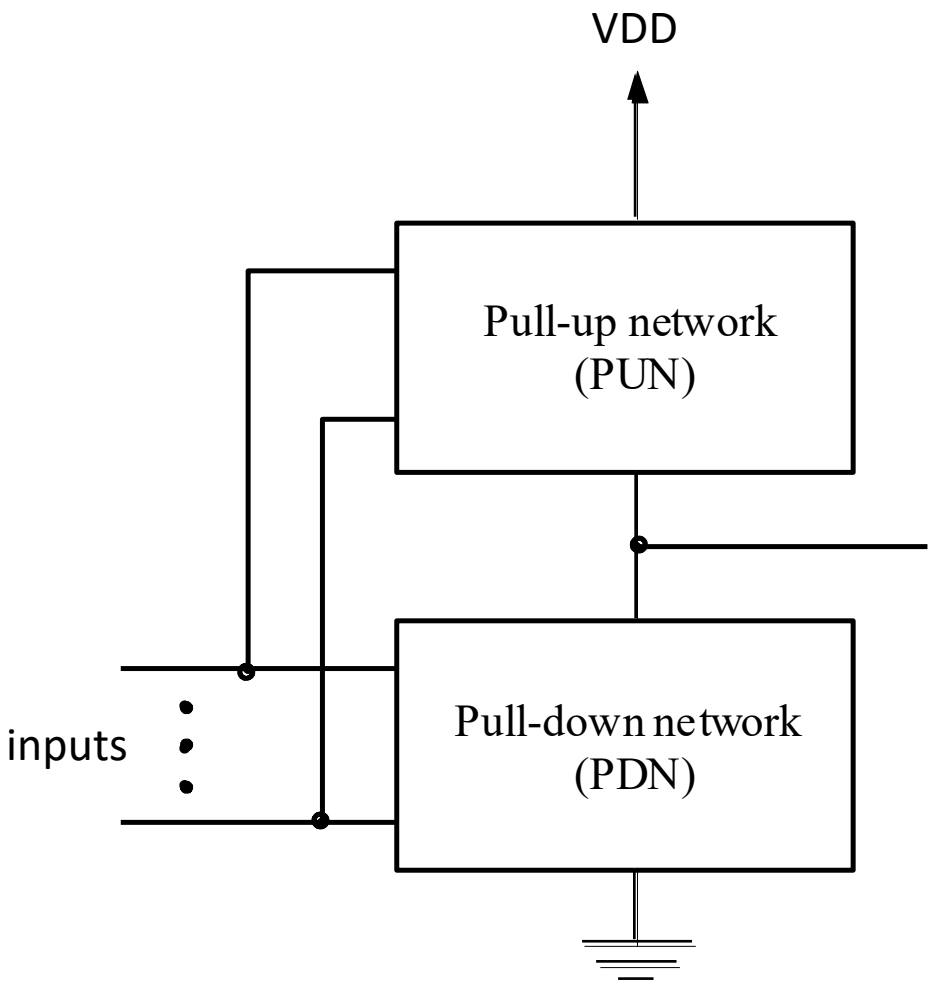


(a) Circuit

x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	0
1	0	off	on	on	off	0
1	1	off	off	on	on	0

(b) Truth table and transistor states

CMOS Implementation Technology – CMOS Principle



PMOS transistors

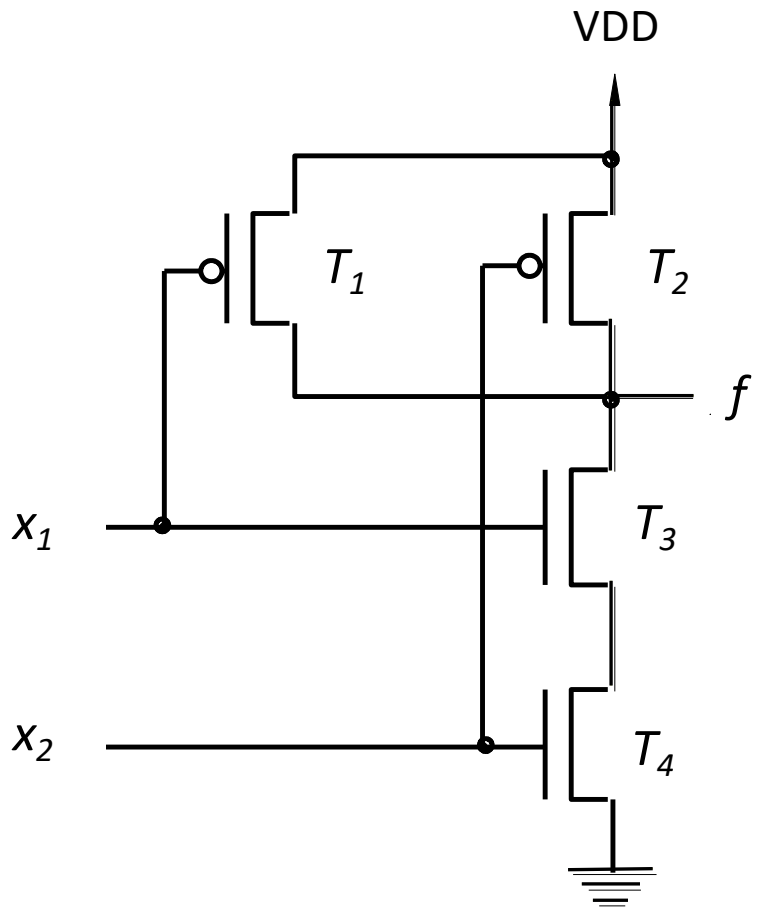
NMOS transistors

(PUN on, PDN off)
or
(PUN off, PDN on)

PUN en PDN have inverted logic behaviour:
$F_{PUN} = F'_{PDN}$

This way:
- no static current
- always connected to V_{DD} or GND

CMOS Implementation Technology – CMOS NAND Gate



(a) Circuit

x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	1
1	0	off	on	on	off	1
1	1	off	off	on	on	0

(b) Truth table and transistor states

CMOS Implementation Technology – Question 2

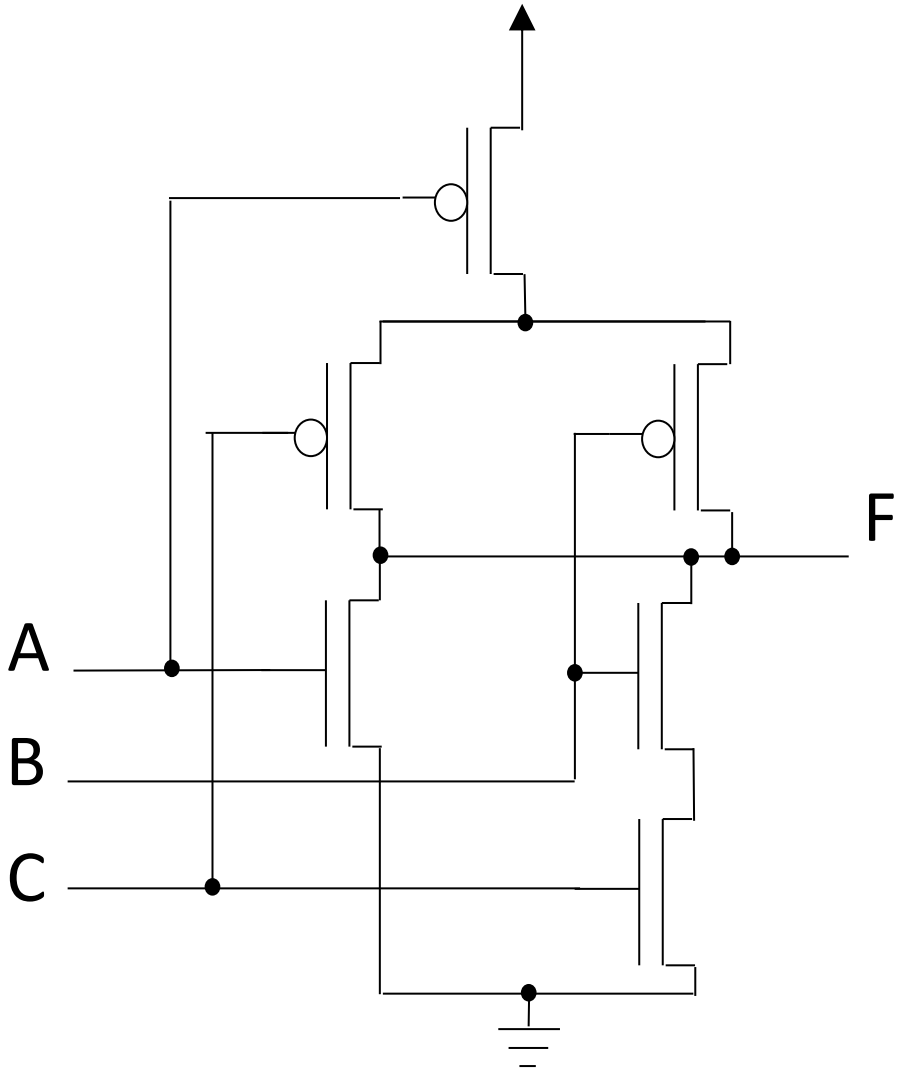
What is an expression for F ?

A. F = A (B + C)

B. F = A + B C

C. F = (A (B + C))’

D. F = (A + B C)’



Path to GND when

A + B C => F = (A + B C)’

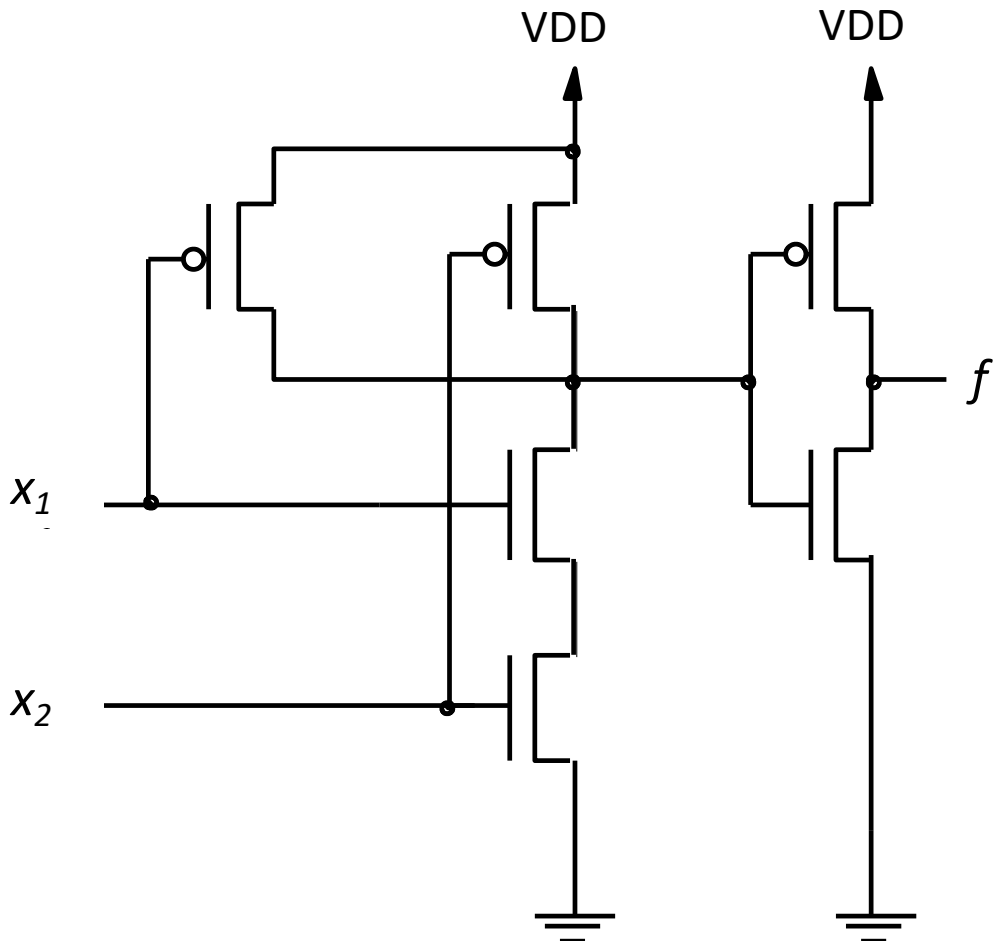
Path to VDD when

A’ (B’ + C’) = A’ (B C)’ = (A + B C)’ => F = (A + B C)’

Hence indeed complementary expressions

F = (A + B C)’ => answer d

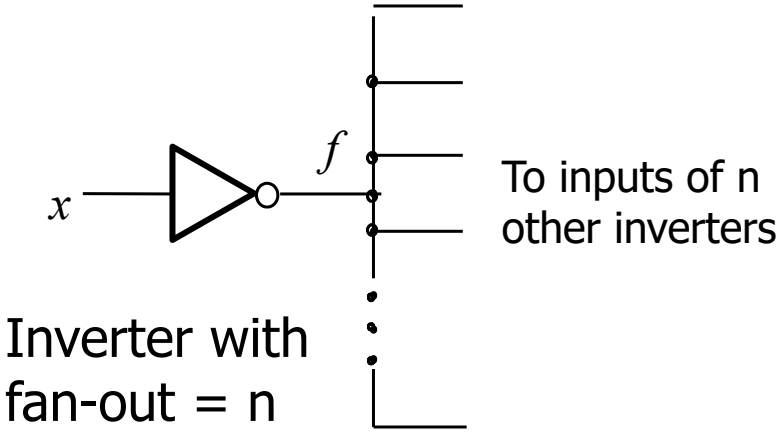
CMOS Implementation Technology – CMOS AND Gate



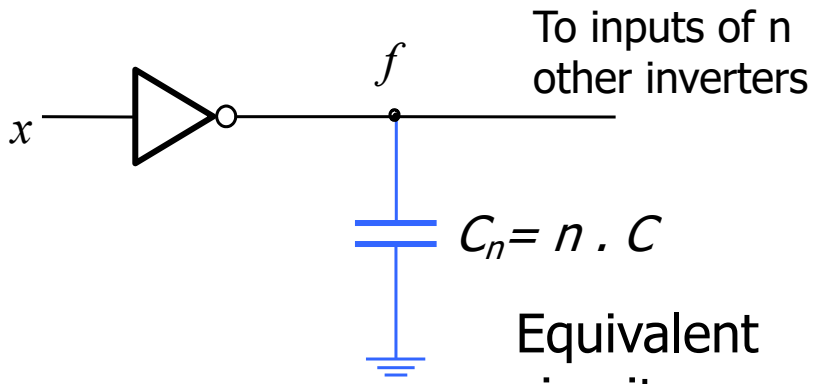
AND has 6 transistors !

while a NAND has only 4 transistors

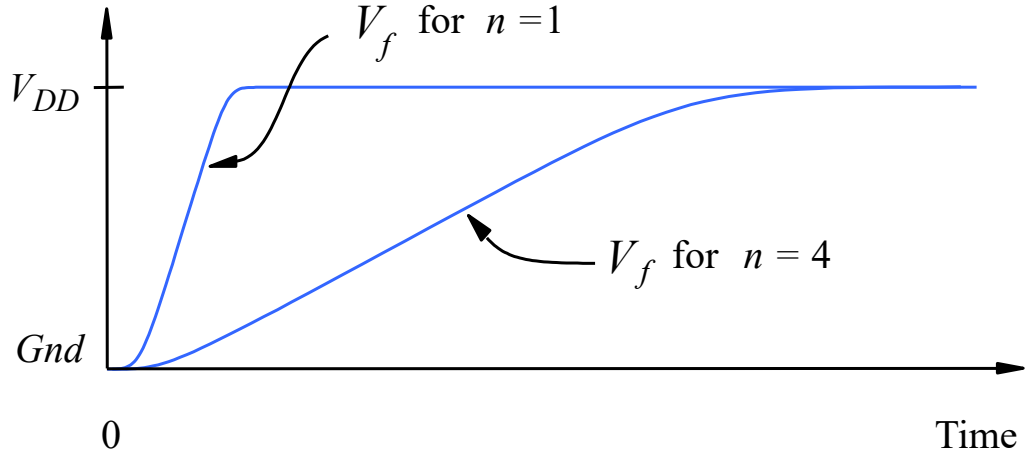
CMOS Implementation Technology – Fan-out



Inverter with
fan-out = n

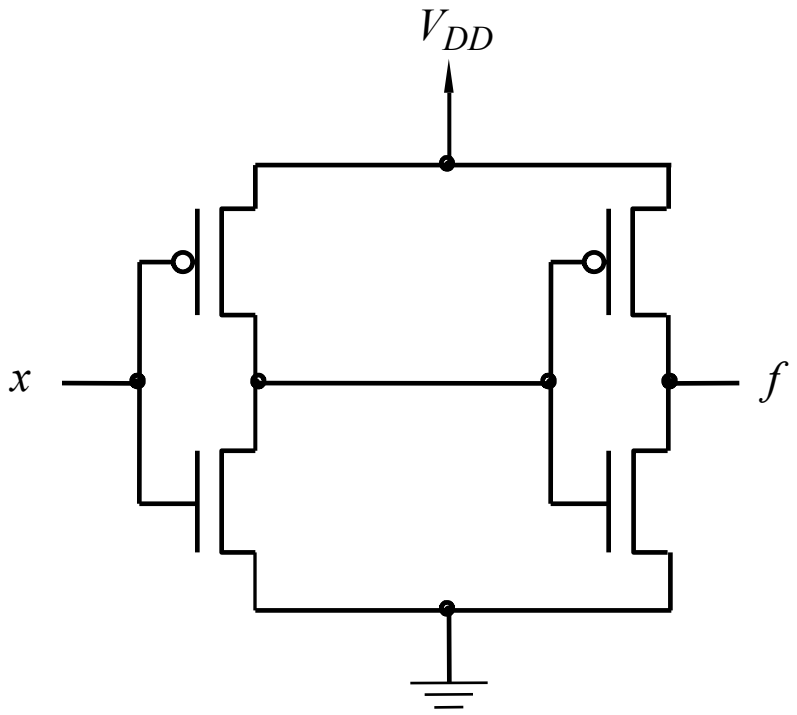


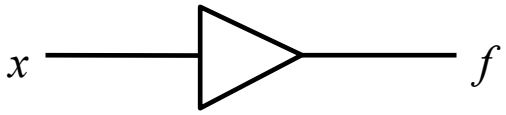
Equivalent
circuit



Delay increases when
fan-out increases

CMOS Implementation Technology – Buffers

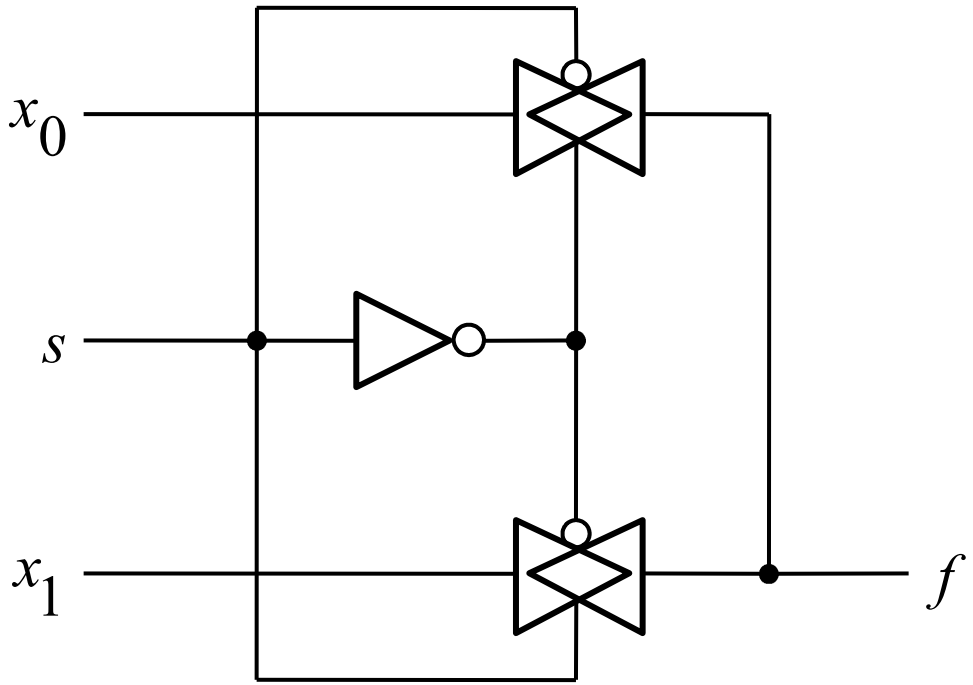




symbol

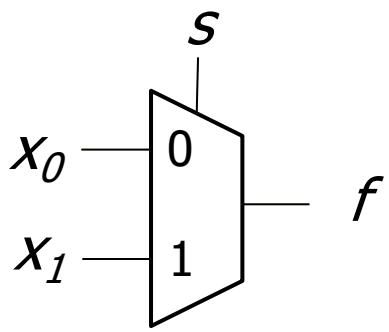
Non-inverting buffer
Transistors of second inverter are wide and can provide much current (to ensure small delay times for high fan-outs).
Despite more levels, total delay can be less.

CMOS Implementation Technology – Multiplexer (or Selector)



s	f
0	x_0
1	x_1

Truth table



symbol

A 2-to-1 multiplexer built with transmission gates.

To do list

- Reading Material book "Digital Design":
  - Sections 1.6, 1.7.4 - 1.7.7, 2.8.

- Assignments Lecture 5



Thank you